



Contents

1. [Motivation](#)
 - 1.1. [Why this is really bad](#)
 - 1.2. [Other Languages](#)
 - 1.2.1. [Rust](#)
 - 1.2.2. [Other Languages](#)
2. [Proposal](#)
 - 2.1. [Change the candidate set for operator lookup](#)
 - 2.2. [Change the meaning of defaulted equality operators](#)
 - 2.3. [Change how we define strong structural equality](#)
 - 2.4. [Change defaulted <=> to also generate a defaulted ==](#)
3. [Important implications](#)
 - 3.1. [Implications for types that have special, but not different, comparisons](#)
 - 3.2. [Implications for comparison categories](#)
4. [Wording](#)
 - 4.1. [Wording for redefining strong structural equality](#)
 - 4.2. [Wording for defaulted <=> generating a defaulted ==](#)
5. [Acknowledgements](#)
6. [References](#)

§ 1. Motivation

P0515 introduced `operator<=>` as a way of generating all six comparison operators from a single function, as well as the ability to default this so as to avoid writing any code at all. See David Stone's [I did not order this!](#) for a very clear, very thorough description of the problem: it does not seem to be possible to implement `<=>` optimally for "wrapper" types. What follows is a super brief run-down.

Consider a type like:

```
struct S {
    vector<string> names;
    auto operator<=>(S const&) const = default;
};
```

Today, this is ill-formed, because `vector` does not implement `<=>`. In order to make this work, we need to add that implementation. It is *not* recommended that `vector` only provide `<=>`, but we will start there and it will become clear why that is the recommendation.

The most straightforward implementation of `<=>` for `vector` is (let's just assume `strong_ordering` and note that I'm deliberately not using `std::lexicographical_compare_3way()` for clarity):

```
template<typename T>
strong_ordering operator<=>(vector<T> const& lhs, vector<T> const& rhs) {
    size_t min_size = min(lhs.size(), rhs.size());
    for (size_t i = 0; i != min_size; ++i) {
        if (auto const cmp = compare_3way(lhs[i], rhs[i]); cmp != 0) {
            return cmp;
        }
    }
    return lhs.size() <=> rhs.size();
}
```

On the one hand, this is great. We wrote one function instead of six, and this function is really easy to understand too. On top of that, this is a really good implementation for `<!` As good as you can get. And our code for `S` works (assuming we do something similar for `string`).

On the other hand, as David goes through in a lot of detail (seriously, read it) this is quite bad for `==`. We're failing to short-circuit early on size differences! If two containers have a large common prefix, despite being different sizes, that's an enormous amount of extra work!

In order to do `==` efficiently, we have to short-circuit and do `==` all the way down. That is:

```

template<typename T>
bool operator==(vector<T> const& lhs, vector<T> const& rhs)
{
    // short-circuit on size early
    const size_t size = lhs.size();
    if (size != rhs.size()) {
        return false;
    }

    for (size_t i = 0; i != size; ++i) {
        // use ==, not <=>, in all nested comparisons
        if (lhs[i] != rhs[i]) {
            return false;
        }
    }

    return true;
}

```

1.1. Why this is really bad

This is really bad on several levels, significant levels.

First, since `==` falls back on `<=>`, it's easy to fall into the trap that once `v1 == v2` compiles and gives the correct answer, we're done. If we didn't implement the efficient `==`, outside of very studious code review, we'd have no way of finding out. The problem is that `v1 <=> v2 == 0` would always give the *correct* answer (assuming we correctly implemented `<=>`). How do you write a test to ensure that we did the short circuiting? The only way you could do it is to time some pathological case - comparing a vector containing a million entries against a vector containing those same million entries plus `1` - and checking if it was fast?

Second, the above isn't even complete yet. Because even if we were careful enough to write `==`, we'd get an efficient `v1 == v2 ...` but still an inefficient `v1 != v2`, because that one would call `<=>`. We would have to also write this manually:

```

template<typename T>
bool operator!=(vector<T> const& lhs, vector<T> const& rhs)
{
    return !(lhs == rhs);
}

```

Third, this compounds *further* for any types that have something like this as a member. Getting back to our `S` above:

```

struct S {
    vector<string> names;
    auto operator<=>(S const&) const = default;
};

```

Even if we correctly implemented `==`, `!=`, and `<=>` for `vector` and `string`, comparing two `S`s for equality *still* calls `<=>` and is *still* a completely silent pessimization. Which *again* we cannot test functionally, only with a timer.

And then, it somehow gets even worse, because it's so easy to fall into yet another trap: you somehow have the diligence to remember that you need to explicitly define `==` for this type and you do it this way:

```

struct S {
    vector<string> names;
    auto operator<=>(S const&) const = default;
    bool operator==(S const&) const = default; // problem solved, right?
};

```

But what does defaulting `operator==` actually do? It [invokes <=>](#). So here's explicit code that seems sensible to add to attempt to address this problem, that does absolutely nothing to address this problem.

The only way to get efficiency is to have every type, even `S` above, implement both not just `<=>` but also `==` and `!=`. By hand.

```

struct S {
    vector<string> names;
    auto operator<=>(S const&) const = default;
    bool operator==(S const& rhs) const { return names == rhs.names; }
    bool operator!=(S const& rhs) const { return names != rhs.names; }
};

```

That is the status quo today and the problem that needs to be solved.

1.2. Other Languages

In order how to best figure out how to solve this problem for C++, it is helpful to look at how other languages have already addressed this issue. While P0515 listed many languages which have a three-way comparison returning a signed integer, there is another set of otherwise mostly-unrelated languages that take a different approach.

1.2.1. Rust

Rust, Kotlin, Swift, Haskell, and Scala are rather different languages in many respects. But they all solve this particular problem in basically the same way: they treat *equality* and *comparison* as separate operations. I want to focus specifically on Rust here as it's arguably the closest language to C++ of the group, but the other three are largely equivalent for the purposes of this specific discussion.

Rust deals in Traits (which are roughly analogous to C++0x concepts and Swift protocols) and it has four relevant Traits that have to do with comparisons:

- `PartialEq` (which is a partial equivalence relation spelled which only requires symmetry and transitivity)
- `Eq` (which extends `PartialEq`, adding reflexivity)
- `PartialOrd` (which allows for incomparability by returning `Option<Ordering>`, where `Ordering` is an enum)
- `Ord` (a total order, which extends `Eq` and `PartialOrd`)

The actual operators are [implicitly generated](#) from these traits, but not all from the same one. Importantly, `x == y` is translated as `PartialEq::eq(x, y)` whereas `x < y` is translated as `PartialOrd::lt(x, y)` (which is effectively checking that `PartialOrd::partial_cmp(x, y)` is `Less`).

That is, you don't get six functions for the price of one. You need to write *two functions*.

Even if you don't know Rust (and I really don't know Rust), I think it would be instructive here would be to look at how the equivalent comparisons are implemented for Rust's `vector` type. The important parts look like this:

<code>Eq</code>	<code>Ord</code>
<pre>impl<A, B> SlicePartialEq for [A] where A: PartialEq { default fn eq(&self, other: &[B]) -> bool { 5 if self.len() != other.len() { 6 return false; 7 } for i in 0..self.len() { 10 if !self[i].eq(&other[i]) { return false; } } true } }</pre>	<pre>impl<A> SliceOrd<A> for [A] where A: Ord { default fn cmp(&self, other: &[A]) -> Ordering { let l = cmp::min(self.len(), other.len()); let lhs = &self[..l]; let rhs = &other[..l]; for i in 0..l { 11 match lhs[i].cmp(&rhs[i]) { Ordering::Equal => (), non_eq => return non_eq, } } self.len().cmp(&other.len()) } }</pre>

In other words, `eq` calls `eq` all the way down while doing short-circuiting whereas `cmp` calls `cmp` all the way down, and these are two separate functions. Both algorithms exactly match our implementation of `==` and `<=>` for `vector` above. Even though `cmp` performs a 3-way ordering, and you can use the result of `a.cmp(b)` to determine that `a == b`, it is *not* the way that Rust (or other languages in this realm like Swift and Kotlin and Haskell) determine equality.

1.2.2. Other Languages

Swift has `Equatable` and `Comparable` protocols. For types that conform to `Equatable`, `!=` is implicitly generated from `==`. For types that conform to `Comparable`, `>`, `>=`, and `<=` are implicitly generated from `<`. Swift does not have a 3-way comparison function.

There are other languages that make roughly the same decision in this regard that Rust does: `==` and `!=` are generated from a function that does equality whereas the four relational operators are generated from a three-way comparison. Even though the three-way comparison *could* be used to determine equality, it is not:

- Kotlin, like Java, has a `Comparable` interface and a separate `equals` method inherited from `Any`. Unlike Java, it has [operator overloading](#): `a == b` means `a?.equals(b) ? : (b === null)` and `a < b` means `a.compareTo(b) < 0`.
- Haskell has the `Data.Eq` and `Data.Ord` type classes. `!=` is generated from `==` (or vice versa, depending on which definition is provided for `Eq`). If a `compare` method is provided to conform to `Ord`, `a < b` means `(compare a b) < 0`.

- Scala's equality operators come from the root `Any` interface, `a == b` means `if (a eq null) b eq null else a.equals(b)`. Its relational operators come from the `Ordered` trait, where `a < b` means `(a compare b) < 0`.

2. Proposal

Fundamentally, we have two sets of operations: equality and comparison. In order to be efficient and not throw away performance, we need to implement them separately. `operator<=>()` as specified in the working draft today generating all six functions just doesn't seem to be a good solution.

This paper proposes to do something similar to the Rust model above and first described in [this section](#) of the previously linked paper: require two separate functions to implement all the functionality.

The proposal has two core components:

- change the candidate set for operator lookup
- change the meaning of defaulted equality operators

And two optional components:

- change how we define [strong structural equality](#), which is important for [P0732R2](#)
- change defaulted `<=>` to also generate a defaulted `==`

2.1. Change the candidate set for operator lookup

Today, lookup for any of the relational and equality operators will also consider `operator<=>`, but preferring [the actual used operator](#).

The proposed change is for the equality operators to *not* consider `<=>` candidates. Instead, inequality will consider equality as a candidate. In other words, here is the proposed set of candidates. There are no changes proposed for the relational operators, only for the equality ones:

Source a @ b	Today (P0515/C++2a)	Proposed
a == b	a == b (a <=> b) == 0 0 == (b <=> a)	a == b b == a
a != b	a != b (a <=> b) != 0 0 != (a <=> b)	a != b !(a == b) !(b == a)
a < b	a < b (a <=> b) < 0 0 < (b <=> a)	
a <= b	a <= b (a <=> b) <= 0 0 <= (b <=> a)	
a > b	a > b (a <=> b) > 0 0 > (b <=> a)	
a >= b	a >= b (a <=> b) >= 0 0 >= (b <=> a)	

In short, `==` and `!=` never invoke `<=>` implicitly.

2.2. Change the meaning of defaulted equality operators

As mentioned earlier, in the current working draft, defaulting `==` or `!=` generates a function that invokes `<=>`. This paper proposes that defaulting `==` generates a member-wise equality comparison and that defaulting `!=` generate a call to negated `==`.

That is:

Sample Code	Meaning Today (P0515/C++2a)	Proposed Meaning
-------------	-----------------------------	------------------

<pre> struct X { A a; B b; C c; auto operator<=>(X const&) const = default; bool operator==(X const&) const = default; bool operator!=(X const&) const = default; }; </pre>	<pre> struct X { A a; B b; C c; ??? operator<=>(X const& rhs) const { if (auto cmp = a <=> rhs.a; cmp != 0) return cmp; if (auto cmp = b <=> rhs.b; cmp != 0) return cmp; return c <=> rhs.c; } bool operator==(X const& rhs) const { return (*this <=> rhs) == 0; } bool operator!=(X const& rhs) const { return (*this <=> rhs) != 0; } }; </pre>	<pre> struct X { A a; B b; C c; ??? operator<=>(X const& rhs) const { if (auto cmp = a <=> rhs.a; cmp != 0) return cmp; if (auto cmp = b <=> rhs.b; cmp != 0) return cmp; return c <=> rhs.c; } bool operator==(X const& rhs) const { return a == rhs.a && b == rhs.b && c == rhs.c; } bool operator!=(X const& rhs) const { return !(*this == rhs); } }; </pre>
--	--	---

These two changes ensure that the equality operators and the relational operators remain segregated.

2.3. Change how we define strong structural equality

[P0732R2](#) relies on *strong structural equality* as the criteria to allow a class to be used as a non-type template parameter - which is based on having a defaulted `<=>` that itself only calls defaulted `<=>` recursively all the way down and has type either `strong_ordering` or `strong_equality`.

This criteria clashes somewhat with this proposal, which is fundamentally about not making `<=>` be about equality. So it would remain odd if, for instance, we rely on a defaulted `<=>` whose return type is `strong_equality` (which itself can never be used to determine actual equality).

We have two options here:

1. Do nothing. Do not change the rules here at all, still require defaulted `<=>` for use as a non-type template parameter. This means that there may be types which don't have a natural ordering for which we would have to both default `==` and default `<=>` (with `strong_equality`), the latter being a function that *only* exists to opt-in to this behavior.
2. Change the definition of strong structural equality to use `==` instead. The wording here would have to be slightly more complex: define a type `T` as having strong structural equality if each subobject recursively has defaulted `==` and none of the subobjects are floating point types.

The impact of this change revolves around the code necessary to write a type that is intended to only be equality-comparable (not ordered) but also usable as a non-type template parameter: only `operator==` would be necessary.

Do nothing	Change definition
<pre> struct C { int i; bool operator==(C const&) const = default; strong_equality operator<=>(C const&) const = default; }; template <C x> struct Z { }; </pre>	<pre> struct C { int i; bool operator==(C const&) const = default; }; template <C x> struct Z { }; </pre>

2.4. Change defaulted `<=>` to also generate a defaulted `==`

One of the important consequences of this proposal is that if you simply want lexicographic, member-wise, ordering for your type - you need to default two functions (`==` and `<=>`) instead of just one (`<=>`):

P0515/C++2a	Proposed
<pre> // all six struct A { auto operator<=>(A const&) const = default; }; // just equality, no relational </pre>	<pre> // all six struct A { bool operator==(A const&) const = default; auto operator<=>(A const&) const = default; }; </pre>

<pre>struct B { strong_equality operator<=>(B const&) const = default; };</pre>	<pre>// just equality, no relational struct B { bool operator==(B const&) const = default; };</pre>
---	---

Arguably, `A` isn't terrible here and `B` is somewhat simpler. But it makes this proposal seem like it's fighting against the promise of P0515 of making a trivial opt-in to ordering.

As an optional extension, this paper proposes that a defaulted `<=>` operator also generate a defaulted `==`. We can do this regardless of whether the return type of the defaulted `<=>` is provided or not, since even `weak_equality` implies `==`.

This change, combined with the core proposal, means that one single defaulted operator is sufficient for full comparison. The difference is that, with this proposal, we still get optimal equality.

This change may also obviate the need for the previous optional extension of changing the definition of strong structural extension. But even still, the changes are worth considering separately.

3. Important implications

This proposal means that for complex types (like containers), we have to write two functions instead of just `<=>`. But we really have to do that anyway if we want performance. Even though the two `vector` functions are very similar, and for `optional` they are even more similar (see below), this seems like a very necessary change.

For compound types (like aggregates), depending on the preference of the previous choices, we either have to default to functions instead or still just default `<=>` ... but we get optimal performance.

Getting back to our initial example, we would write:

```
struct S {
    vector<string> names;
    bool operator==(S const&) const = default; // (*) if 2.4 not adopted
    auto operator<=>(S const&) const = default;
};
```

Even if we choose to require defaulting `operator==` in this example, the fact that `<=>` is no longer considered as a candidate for equality means that the worst case of forgetting this function is that equality *does not compile*. That is a substantial improvement over the alternative where equality compiles and has subtly worse performance that will be very difficult to catch.

3.1. Implications for types that have special, but not different, comparisons

There are many kinds of types for which the defaulted comparison semantics are incorrect, but nevertheless don't have to do anything different between equality and ordering. One such example is `optional<T>`. Having to write two functions here is extremely duplicative:

P0515/C++2a	Proposed
<pre>template <typename T, typename U> constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs) const -> decltype(compare_3way(*lhs, *rhs)) { if (lhs.has_value() && rhs.has_value()) { return compare_3way(*lhs, *rhs); } else { return lhs.has_value() <=> rhs.has_value(); } }</pre>	<pre>template <typename T, typename U> constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs) const 4 -> decltype(compare_3way(*lhs, *rhs)) { if (lhs.has_value() && rhs.has_value()) { 7 return compare_3way(*lhs, *rhs); } else { 9 return lhs.has_value() <=> rhs.has_value(); } } template <typename T, typename U> constexpr auto operator==(optional<T> const& lhs, optional<U> const& rhs) const 16 -> decltype(*lhs == *rhs) { if (lhs.has_value() && rhs.has_value()) { 19 return *lhs == *rhs; } else { 21 return lhs.has_value() == rhs.has_value(); } }</pre>

As is probably obvious, the implementations of `==` and `<=>` are basically identical: the only difference is that `==` calls `==` and `<=>` calls `<=>` (or really `compare_3way`). It may be very tempting to implement `==` to just call `<=>`, but that would be wrong! It's critical that `==` call `==` all the way down.

It's important to keep in mind three things.

1. In C++17 we'd have to write six functions, so writing two is a large improvement.
2. These two functions may be duplicated, but they give us optimal performance - writing the one `<=>` to generate all six comparison functions does not.
3. The amount of special types of this kind - types that have non-default comparison behavior but perform the same algorithm for both `==` and `<=>` - is fairly small. Most container types would have separate algorithms. Typical types default both, or just default `==`. The canonical examples that would need special behavior are `std::array` and `std::forward_list` (which either have fixed or unknown size and thus cannot short-circuit) and `std::optional` and `std::variant` (which can't do default comparison). So this particular duplication is a fairly limited problem.

§ 3.2. Implications for comparison categories

One of the features of P0515 is that you could default `<=>` to, instead of returning an order, simply return some kind of equality:

```
struct X {
    std::strong_equality operator<=>(X const&) const = default;
};
```

In a world where neither `==` nor `!=` would be generated from `<=>`, this no longer makes much sense. We could have to require that the return type of `<=>` be some kind of ordering - that is, at least `std::partial_ordering`. Allowing the declaration of `X` above would be misleading, at best.

This means there may not be a way to differentiate between `std::strong_equality` and `std::weak_equality`. The only other place to do this kind of differentiation would be if we somehow allowed it in the return of `operator==`:

```
struct X {
    std::strong_equality operator==(X const&) const = default;
};
```

And I'm not sure this makes any sense.

§ 4. Wording

What follows is the wording from the core sections of the proposal (2.1 and 2.2).

Change 10.10.3 [class.rel.eq] paragraph 2:

The [relational](#) operator function with parameters `x` and `y` is defined as deleted if

- overload resolution ([over.match]), as applied to `x <=> y` (also considering synthesized candidates with reversed order of parameters ([over.match.oper])), results in an ambiguity or a function that is deleted or inaccessible from the operator function, or
- the operator `@` cannot be applied to the return type of `x <=> y` or `y <=> x`.

Otherwise, the operator function yields `x <=> y @ 0` if an operator `<=>` with the original order of parameters was selected, or `0 @ y <=> x` otherwise.

Add a new paragraph after 10.10.3 [class.rel.eq] paragraph 2:

The return value `V` of type `bool` of the defaulted `==` (equal to) operator function with parameters `x` and `y` of the same type is determined by comparing corresponding elements `xi` and `yi` in the expanded lists of subobjects (class spaceship) for `x` and `y` until the first index `i` where `xi == yi` yields a value result which, contextually converted to `bool`, yields `false`. If no such index exists, `V` is `true`. Otherwise, `V` is `false`.

Add another new paragraph after 10.10.3 [class.rel.eq] paragraph 2:

The `!=` (not equal to) operator function with parameters `x` and `y` is defined as deleted if

- overload resolution ([over.match]), as applied to `x == y` (also considering synthesized candidates with reversed order of parameters ([over.match.oper])), results in an ambiguity or a function that is deleted or inaccessible from the operator function, or
- the negation operator cannot be applied to the return type of `x == y` or `y == x`.

Otherwise, the `!=` operator function yields `!(x == y)`, if an operator `==` with the original order of parameters was selected, or `!(y == x)` otherwise.

Change the example in [class.rel.eq] paragraph 3:

```

struct C {
    friend std::strong_equality operator<=>(const C&, const C&);
    friend bool operator==(const C& x, const C& y) = default; // OK, returns x <=> y == 0
    bool operator<(const C&) = default; // OK, function is deleted
    bool operator!=(const C&) = default; // OK, function is deleted
};

struct D {
    int i;
    friend bool operator==(const D& x, const D& y) const = default; // OK, returns x.i == y.i
    bool operator!=(const D& z) const = default; // OK, returns !(*this == z).
};

```

Change 11.3.1.2 [over.match.oper] paragraph 3.4:

For the relational ([`expr.rel`]) **and equality** ([`expr.eq`]) operators, the rewritten candidates include all member, non-member, and built-in candidates for the operator `<=>` for which the rewritten expression `(x <=> y) @ 0` is well-formed using that operator `<=>`. For the relational ([`expr.rel`]), **equality** ([`expr.eq`]), and three-way comparison ([`expr.spaceship`]) operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each member, non-member, and built-in candidate for the operator `<=>` for which the rewritten expression `0 @ (y <=> x)` is well-formed using that operator `<=>`. For the `!=` (not equal to) operator ([`expr.eq`]), the rewritten candidates include all member, non-member, and built-in candidates for the operator `==` for which the rewritten expression `!(x == y)` is well-formed using that operator `==`. For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each member, non-member, and built-in candidate for the operator `==` for which the rewritten expression `(y == x) @ true` is well-formed using that operator `==`. [Note: A candidate synthesized from a member candidate has its implicit object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. —end note] In each case, rewritten candidates are not considered in the context of the rewritten expression. For all other operators, the rewritten candidate set is empty.

4.1. Wording for redefining strong structural equality

Replace 10.10.1 [class.compare.default] paragraph 2:

~~A three-way comparison operator for a class type `C` is a structural comparison operator if it is defined as defaulted in the definition of `C`, and all three-way comparison operators it invokes are structural comparison operators. A type `T` has strong structural equality if, for a glvalue `x` of type `const T`, `x <=> x` is a valid expression of type `std::strong_ordering` or `std::strong_equality` and either does not invoke a three-way comparison operator or invokes a structural comparison operator.~~

with:

An `==` (equal to) operator is a structural equality operator if:

- it is a built-in candidate ([`over.built`]) where neither argument has floating point type, or
- it is an operator for a class type `C` that is defined as defaulted in the definition of `C` and all `==` operators it invokes are structural equality operators.

A type `T` has strong structural equality if, for a glvalue `x` of type `const T`, `x == x` is a valid expression of type `bool` and invokes a structural equality operator.

4.2. Wording for defaulted `<=>` generating a defaulted `==`

Add to 10.10.3 [class.rel.eq], below the description of defaulted `==`:

If the class definition does not explicitly declare an `==` (equal to) operator function ([`expr.eq`]) and declares a defaulted three-way comparison operator function ([`class.spaceship`]) that is not defined as deleted, a defaulted `==` operator function is declared *implicitly*. The implicitly-declared `==` operator for a class `X` will have the form

```
bool X::operator==(const X&, const X&);
```

and will follow the rules described above.

5. Acknowledgements

This paper most certainly would not exist without David Stone's extensive work in this area. Thanks also to Agustín Bergé for discussing issues with me.

6. References

- [P0515R3] "Consistent comparison" by Herb Sutter, Jens Maurer, Walter E. Brown, 2017-11-10
- [P0732R2] "Class Types in Non-Type Template Parameters" by Jeff Snyder, Louis Dionne, 2018-06-06
- [haskell.eq] Data.Eq - Haskell documentation

- [\[haskell.ord\]](#) Data.Ord - Haskell documentation
- [\[kotlin.any\]](#) Any - Kotlin Programming Language
- [\[kotlin.comp\]](#) Comparable - Kotlin Programming Language
- [\[kotlin.oper\]](#) Operator overloading - Kotlin Programming Language
- [\[revzin.impl\]](#) "Implementing the spaceship operator for optional" by Barry Revzin, 2017-11-16
- [\[rust.eq\]](#) Implementation of Eq for Slice
- [\[rust.oper\]](#) Comparison Operators - The Rust Reference
- [\[rust.ord\]](#) Implementation of Ord for Slice
- [\[scala.any\]](#) Scala Standard Library - Any
- [\[scala.ord\]](#) Scala Standard Library - Ordered
- [\[stepanov.fgpp\]](#) "Fundamentals of Generic Programming" by James C. Dehnert and Alexander Stepanov, 1998
- [\[stone.order\]](#) "I did not order this! Why is it on my bill?" by David Stone, 2018-08-06
- [\[swift.comp\]](#) Comparable - Swift Standard Library
- [\[swift.eq\]](#) Equatable - Swift Standard Library